OS_ORANGE_PROBLEM_U4

NAME : D.R.Rushmitha

SRN : PES1UG24AM910

SECTION : E

Phase 1: Object Storage Foundation

Screenshot 1A: Output of ./test_objects showing all tests passing

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the shared directory structure.

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2: Tree Objects

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d
4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

Phase 3: The Index (Staging Area)

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ echo "hello" > file1.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ echo "world" > file2.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes add file1.txt file2.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  tree.h
  untracked:  pes.h
  untracked:  tree.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  object.c
  untracked:  commit.h
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  index.h
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776322946 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776322953 6 file2.txt
```

Phase 4: Commits and History

Screenshot 4A: **Output of ./pes log showing three commits with hashes, authors, timestamps, and messages**.

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ echo "Hello" > hello.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes add hello.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes commit -m "Initial commit"
Committed: eadf6b1aa56d... Initial commit
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ echo "World" >> hello.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes add hello.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes commit -m "Add world"
Committed: 5bfee6806746... Add world
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ echo "Goodbye" > bye.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes add bye.txt
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 12783cba7f1f... Add farewell
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ ./pes log
commit 12783cba7f1fa1835388ef8a5debb379b3e86032b29931f2fbd8df73d7886311
Author: Rushmitha <PES1UG24AM910>
Date:   1776330290

    Add farewell


commit 5bfee68067466342feefb10b03eba2c6b52b03d2475c80b95efdcde020a79104
Author: Rushmitha <PES1UG24AM910>
Date:   1776330267

    Add world


commit eadf6b1aa56db2e2426e9535b73aa21a78ea6aa7003241838d87a4b0313c746f
Author: Rushmitha <PES1UG24AM910>
Date:   1776330249

    Initial commit
```

Screenshot 4B: **find .pes -type f | sort showing object store growth after three commits.**

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/12/783cba7f1fa1835388ef8a5debb379b3e86032b29931f2fbd8df73d7886311
.pes/objects/5b/fee68067466342feefb10b03eba2c6b52b03d2475c80b95efdcde020a79104
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ea/df6b1aa56db2e2426e9535b73aa21a78ea6aa7003241838d87a4b0313c746f
.pes/refs/heads/main
```

Screenshot 4C: **cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.**

```
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ cat .pes/refs/heads/main
12783cba7f1fa1835388ef8a5debb379b3e86032b29931f2fbd8df73d7886311
vboxuser@Rushmitha-AM910:~/PES1UG24AM910-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans :

A branch in PES (like Git) is just a file inside .pes/refs/heads/ containing a commit hash.

A branch in PES is just a file inside .pes/refs/heads/ that stores a commit hash.

i) What files need to change in .pes/?

- The main file that changes is .pes/HEAD
- It should be updated to point to the new branch:
- ref: refs/heads/<branch>

ii) What must happen to the working directory?

- Read the commit hash from .pes/refs/heads/<branch>
- Load the corresponding commit object
- Get the tree (snapshot of files) from that commit
- Update the working directory so it matches the tree:
    - create new files
    - overwrite modified files
    - delete files that are not present in the new branch

iii) What makes this operation complex?

- We must be careful not to overwrite any user's uncommitted changes
- Need to compare current files with the target branch files
- Handle file deletions and directories properly
- It involves coordination between:
    - index
    - object store
    - working directory

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans :

We must prevent checkout if the user has uncommitted changes that may get overwritten.

How to detect this:

- First compare the index with the working directory
  - For each file in the index:
    - check file metadata like size or modification time
    - or recompute the hash
  - If there is a difference, it means the file is modified locally
- Then compare the target branch tree with the index
  - If the same file has different content in the target branch, it can cause a conflict

Condition for refusing checkout:

- If a file is modified in the working directory
- AND the same file is different in the target branch

Then checkout should be stopped to avoid losing changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans :

A detached HEAD means that instead of HEAD pointing to a branch, it directly points to a commit.

Normally:
HEAD → branch → commit

In detached state:
HEAD → commit

When we make commits in this state, new commits are still created, but they are not connected to any branch. So no branch is pointing to these commits.

The problem is that these commits are not referenced anywhere, so they can be lost later during garbage collection.

To recover them, we can:

- create a new branch at that commit (pes branch <name>)
- or manually store the commit hash inside .pes/refs/heads/<branch>

So basically, detached HEAD is useful for temporary work, but commits must be saved to a branch if we don’t want to lose them.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans :

Over time, the object store keeps growing because it stores all blobs, trees and commits, even the ones that are no longer used. So we need a way to remove objects that are not reachable from any branch.

The basic idea is:

- Start from all branch heads in .pes/refs/heads/
- From each commit, follow its parent commits
- From each commit, also go to its tree
- From the tree, go to blobs (files) and subtrees
- Mark all these objects as **reachable**

After this, go through all objects in .pes/objects/:

- If an object is not marked as reachable, delete it

To keep track of reachable objects efficiently, we can use a hash set.
This allows fast checking (O(1)) to see whether an object has already been visited.

For a repository with 100,000 commits and 50 branches:

- We start from 50 branch heads
- But most commits share history, so we don’t visit 50 × 100,000
- Instead, we visit roughly:
    - ~100,000 commits
    - ~100,000 trees
    - and many blobs (depending on files)
    - So overall, we might visit a few hundred thousand objects (not millions in most cases).

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this? can you give me as a normal student answer but organised.

Ans :

- Running garbage collection (GC) during a commit is risky because both operations use the object store at the same time.
- During a commit:
  - new objects (blobs, trees, commits) are created
  - but they are not immediately linked to any branch
- Garbage collection checks only objects that are reachable from branches.
- So, if GC runs at the same time:
  - it may think these new objects are unused
  - and delete them
- This can cause problems because:
  - the commit may later try to use those deleted objects
  - leading to corruption or missing data
- Git avoids this problem by:
  - writing objects first, then updating references
  - using locking to avoid conflicts
  - not deleting recently created objects immediately
  - running GC in a safe, controlled way .